Dossier d'Architecture Technique & Manuel d'Ingénierie (DAT)

Système Planner-Pro — Operating System de Productivité & Ingénierie

Statut : Production Ready

Version : 2.0 (Spécifications Techniques Approfondies)

Public cible : Architectes Logiciels, Ingénieurs Backend/Frontend, DevOps & Lead Developers

Dernière révision : Octobre 2026

Sommaire Exécutif & Table des Matières

1. [Vision Architecturale & Principes Directeurs](#1-vision-architecturale--principes-directeurs)

2. [Architecture Globale des Conteneurs (Modèle C4 Niveau 2)](#2-architecture-globale-des-conteneurs-modèle-c4-niveau-2)

3. [Authentification, Chiffrement Symétrique & Matrice RBAC](#3-authentification-chiffrement-symétrique--matrice-rbac)

4. [Protocole Cryptographique des Invitations & Adhésion](#4-protocole-cryptographique-des-invitations--adhésion)

5. [Moteur Kanban & Algorithme Acyclique Gantt (DFS)](#5-moteur-kanban--algorithme-acyclique-gantt-dfs)

6. [Time-Tracking & Passerelle WebSockets Temps Réel](#6-time-tracking--passerelle-websockets-temps-réel)

7. [Calendrier Interactif & Algorithme de Time-Blocking](#7-calendrier-interactif--algorithme-de-time-blocking)

8. [Bloc-notes Intelligent & Moteur Lexical (Notes-to-Tasks)](#8-bloc-notes-intelligent--moteur-lexical-notes-to-tasks)

9. [Moteur Financier, Budgets & Burn Rate Projet](#9-moteur-financier-budgets--burn-rate-projet)

10. [Synchronisation Hybride Asynchrone : Transactional Outbox vers Cloud Firestore](#10-synchronisation-hybride-asynchrone--transactional-outbox-vers-cloud-firestore)

11. [Infrastructure de Production, Déploiement & Résilience](#11-infrastructure-de-production-déploiement--résilience)

---

1. Vision Architecturale & Principes Directeurs

1.1 Contexte & Paradigme "Focus & Flow"

Planner-Pro n'est pas une simple application de gestion de tickets de plus (CRUD). Son architecture est conçue pour éliminer la friction cognitive entre la réflexion libre (prise de notes), la formalisation de projet (Kanban, Gantt, jalons) et l'exécution minutée (chronomètre temps réel, Time-blocking, Pomodoro).

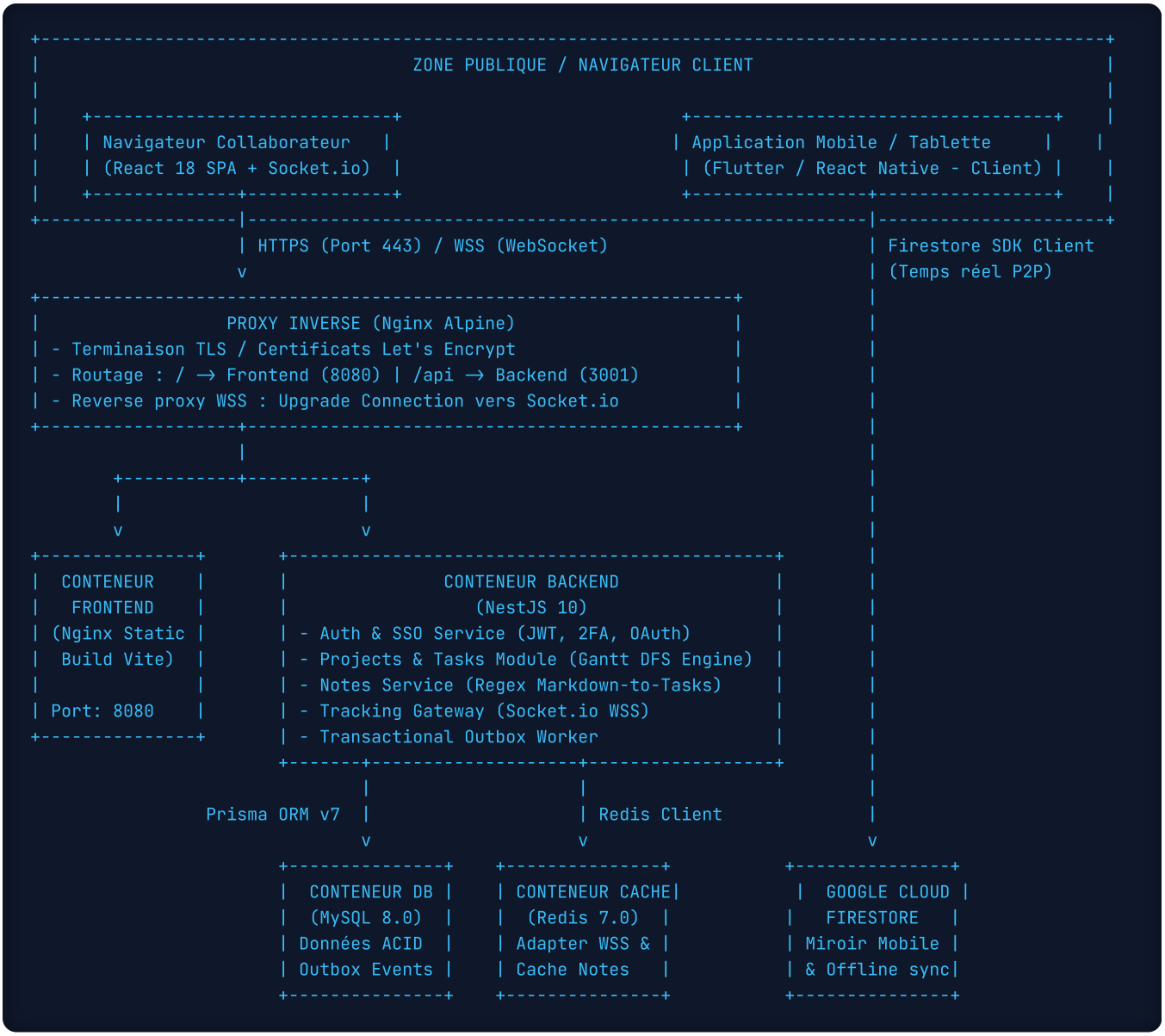
1.2 Principes d'Ingénierie Logicielle

- **Architecture Modulaire DDD (Domain-Driven Design)** : Découpage strict du backend NestJS en domaines d'affaires autonomes (`auth` , `projects` , `tracking` , `notes` , `notifications` , `firestore-sync`).
- **Zéro Dépendance Circulaire Métier & Graphe Acyclique Dirigé (DAG)** : Les dépendances de tâches sont modélisées mathématiquement comme un DAG vérifié par recherche en profondeur (DFS).
- **Sécurité Défensive & Zéro Exposition** : Tout secret externe est chiffré symétriquement en AES-256-GCM. Aucun token brut d'invitation n'est persistant en base de données.
- **Résilience aux Pannes Réseau (Double-Write Problem résolu)** : L'intégration avec des services externes ou Cloud distants (comme Firestore) n'utilise jamais d'appel synchrone dans la transaction métier, mais s'appuie sur le patron *Transactional Outbox*.
- **Typage de bout en bout** : Monorepo TypeScript strict avec validation de runtime (Pipes `class-validator` , Zod/Typesafe Prisma Client).

---

2. Architecture Globale des Conteneurs (Modèle C4 Niveau 2)

Le système est conteneurisé et orchestré via Docker Compose, séparant hermétiquement les couches de présentation, de traitement d'affaires et de persistance.



2.1 Description Technique des Conteneurs

Conteneur 1 : Proxy Inverse Nginx (`ekose-nginx-1` ou Nginx Hôte)

- **Fonctions** : Terminaison TLS (Certificats RSA 4096 / ECDSA Let's Encrypt), négociation HTTP/2, compression Gzip/Brotli, rate-limiting (60 req/min sur endpoints sensibles), et négociation du tunnel WebSocket (`proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"`).
- **Réseau** : Expose le port 80 (redirection permanente HTTP 301 vers 443) et le port 443.

Conteneur 2 : Frontend SPA (`planner-pro-production-frontend-1`)

- **Framework & Build** : React 18, TypeScript, bundler Vite.
- **Moteurs de rendu & Interaction** : `@dnd-kit/core` et `@dnd-kit/sortable` pour le glisser-déposer haute vélocité (Kanban, Calendrier, Sprints), Lucide React pour l'iconographie, Context API (`AppContext`) pour la distribution réactive de l'état global.
- **Serveur interne** : Nginx 1.27 Alpine servant les assets statiques compilés avec politique de cache agressive (`Cache-Control: public, max-age=31536000, immutable` pour les chunks hashés et `no-cache` pour `index.html`).

Conteneur 3 : Backend API NestJS (`planner-pro-production-backend-1`)

- **Framework & Moteur** : Node.js 20 LTS, NestJS 10, architecture modulaire en injection de dépendances.
- **Passerelle Temps Réel** : `@nestjs/websockets` avec moteur Socket.io v4. Couplé à `@socket.io/redis-adapter` pour permettre un dimensionnement horizontal multi-instances sans perte de cohésion de sessions.
- **Couche d'Accès aux Données** : Prisma ORM v7 avec découpage multi-fichiers de schéma (`auth.prisma`, `project.prisma`, `tracking.prisma`, `notes.prisma`). Pool de connexions optimisé avec timeouts stricts.

Conteneur 4 : Base de Données Principale (`planner-pro-production-db-1`)

- **Moteur SGBDR** : MySQL 8.0 Community Edition configuré en encodage `utf8mb4_unicode_ci`.
- **Garanties** : Moteur InnoDB, transactions au niveau d'isolation `READ COMMITTED`, clés étrangères avec cascades contrôlées, indexation B-Tree sur les identifiants de partitionnement métier (`workspaceId`, `projectId`, `userId`).

Conteneur 5 : Cache & Pub/Sub Distribué (`planner-pro-production-cache-1`)

- **Moteur** : Redis 7.0 Alpine.
- **Utilisations** :
  1. Adaptateur Pub/Sub pour la diffusion de messages WebSocket entre workers Node.js.
  2. Cache à faible latence pour la lecture des notes (`TTL: 3600s`).
  3. Stockage volatile des états de présence et timers actifs en cours de session.

Conteneur 6 : Cloud Firestore (Google Cloud Platform)

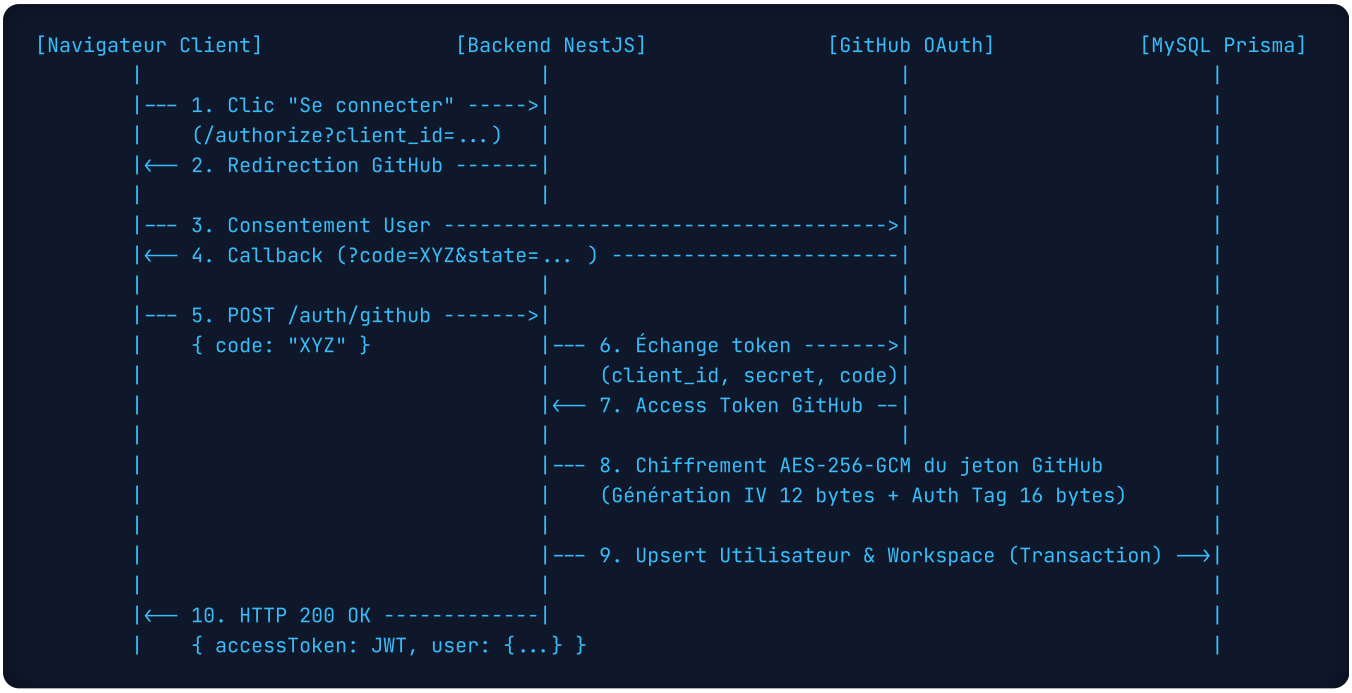
- **Rôle** : Base de données NoSQL distribuée servant de miroir temps réel pour les applications mobiles et de secours hors-ligne (Offline-First sync). Alimenté de manière asynchrone et idempotente par le worker *Transactional Outbox* du backend.

---

3. Authentification, Chiffrement Symétrique & Matrice RBAC

3.1 Protocole d'Authentification OAuth 2.0 GitHub & SSO

Le flux d'authentification OAuth 2.0 s'exécute selon la spécification RFC 6749 :



3.2 Spécification du Chiffrement Symétrique AES-256-GCM

Pour sécuriser les jetons d'accès tiers (GitHub personal tokens, jetons d'intégration Google Calendar, etc.), le service `encryption.util.ts` implémente un chiffrement symétrique authentifié **AES-256-GCM** (Galois/Counter Mode) :

- **Génération de la clé** : Clé symétrique de 256 bits dérivée de la variable d'environnement `ENCRYPTION_KEY`.
- **Vecteur d'Initialisation (IV)** : 12 octets pseudo-aléatoires cryptographiques (`crypto.randomBytes(12)`), unique par opération de chiffrement.
- **Tag d'Authentification (Auth Tag)** : 16 octets générés par l'algorithme GCM, garantissant l'intégrité et la non-altération du payload chiffré.
- **Format de sérialisation en base** : `::`. Toute tentative de falsification du ciphertext provoque le rejet immédiat du déchiffrement (`CipherJobException`).

3.3 Matrice des Permissions RBAC à Deux Niveaux

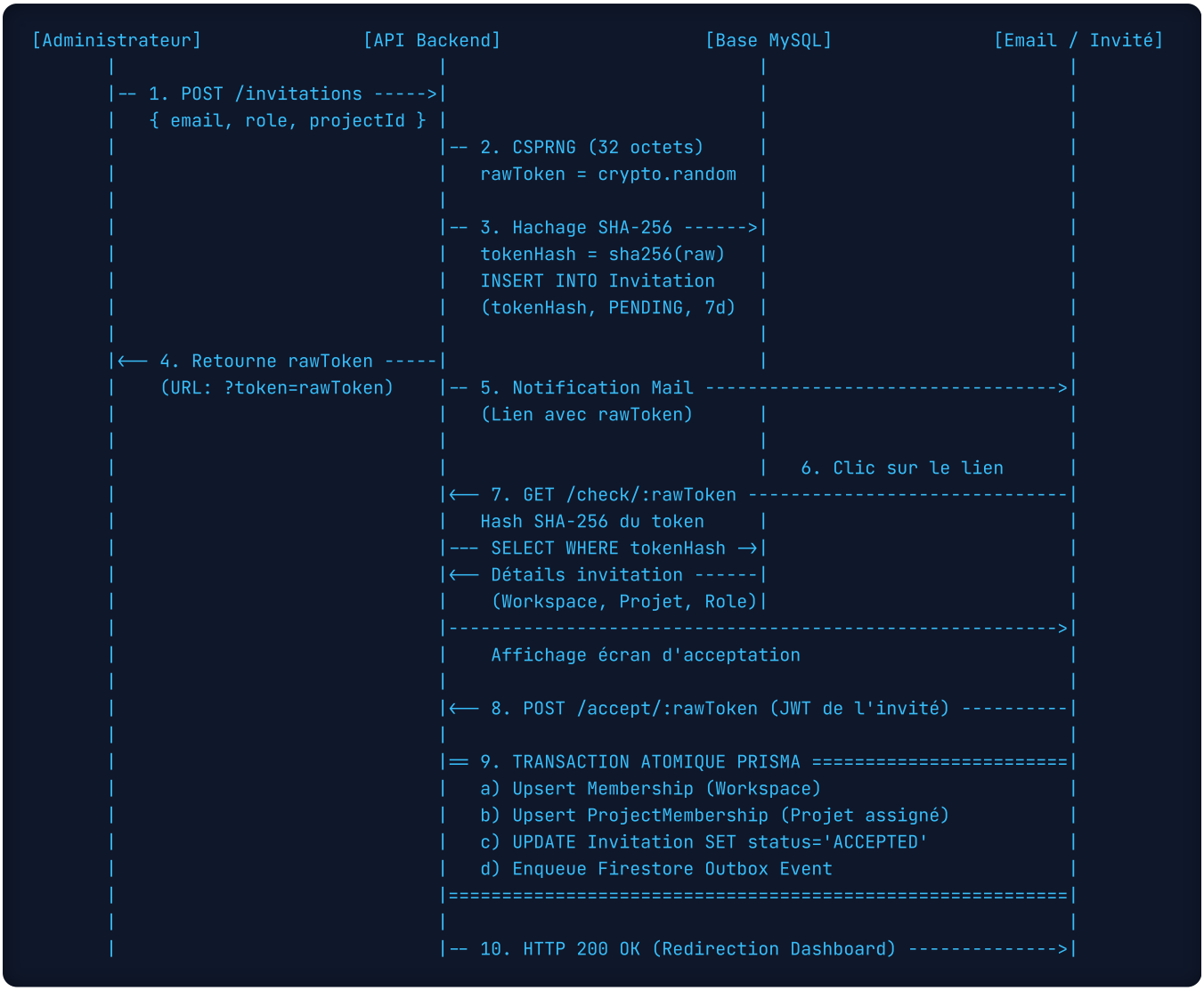
Planner-Pro sépare la gouvernance de l'organisation générale (Niveau Workspace) de la gouvernance opérationnelle d'un projet spécifique (Niveau Projet).

```
=================================================================================================================
                              MATRICE DES HABILITATIONS WORKSPACE & PROJET
=================================================================================================================
Capacité Technique / Fonctionnelle        | OWNER | ADMIN | MEMBER | VIEWER | MANAGER | CONTRIBUTOR | CLIENT
------------------------------------------+-------+-------+--------+--------+---------+-------------+-------
Supprimer ou renommer l'Espace de travail |  OUI  |  NON  |  NON   |  NON   |   -     |     -       |  -
Gérer les intégrations & Webhooks globaux |  OUI  |  OUI  |  NON   |  NON   |   -     |     -       |  -
Inviter un membre au niveau Espace        |  OUI  |  OUI  |  NON   |  NON   |   -     |     -       |  -
Créer un nouveau projet                   |  OUI  |  OUI  |  OUI   |  NON   |   -     |     -       |  -
Modifier les paramètres / Clôturer Projet |  OUI* |  OUI* |   -    |   -    |   OUI   |    NON      |  NON
Créer, éditer, déplacer des tâches        |  OUI* |  OUI* |   -    |   -    |   OUI   |    OUI      |  NON
Lancer un chronomètre (Time-Tracking)     |  OUI* |  OUI* |   -    |   -    |   OUI   |    OUI      |  NON
Modifier les taux horaires & finances     |  OUI  |  OUI  |  NON   |  NON   |   OUI   |    NON      |  NON
Consulter les rapports financiers & marge |  OUI  |  OUI  |  NON   |  NON   |   OUI   |    NON      |  NON
Commenter une tâche                       |  OUI  |  OUI  |  OUI   |  NON   |   OUI   |    OUI      |  OUI
Consulter le projet en lecture seule      |  OUI  |  OUI  |  OUI   |  OUI   |   OUI   |    OUI      |  OUI
=================================================================================================================
* Note : Les rôles Workspace OWNER et ADMIN héritent implicitement de tous les droits MANAGER sur
l'ensemble des sous-projets de leur espace via le guard `assertProjectRole()`.
```

---

4. Protocole Cryptographique des Invitations & Adhésion

Le système d'invitation garantit qu'aucun lien d'invitation ne puisse être deviné, falsifié ou récupéré en cas de fuite de base de données.



4.1 Détails Techniques de Sécurité

1. **Entropie Cryptographique** : Le jeton brut `rawToken` est généré par `crypto.randomBytes(32).toString('hex')`, offrant un espace d'états de 2^{256}, éliminant tout risque d'attaque par force brute ou collision.

2. **Hachage Unidirectionnel en Base** : Seul le digest `crypto.createHash('sha256').update(rawToken).digest('hex')` est stocké en base de données. Même avec un accès administrateur direct à la table MySQL `invitations`, un attaquant ne peut pas reconstituer le lien magique à présenter à l'API.

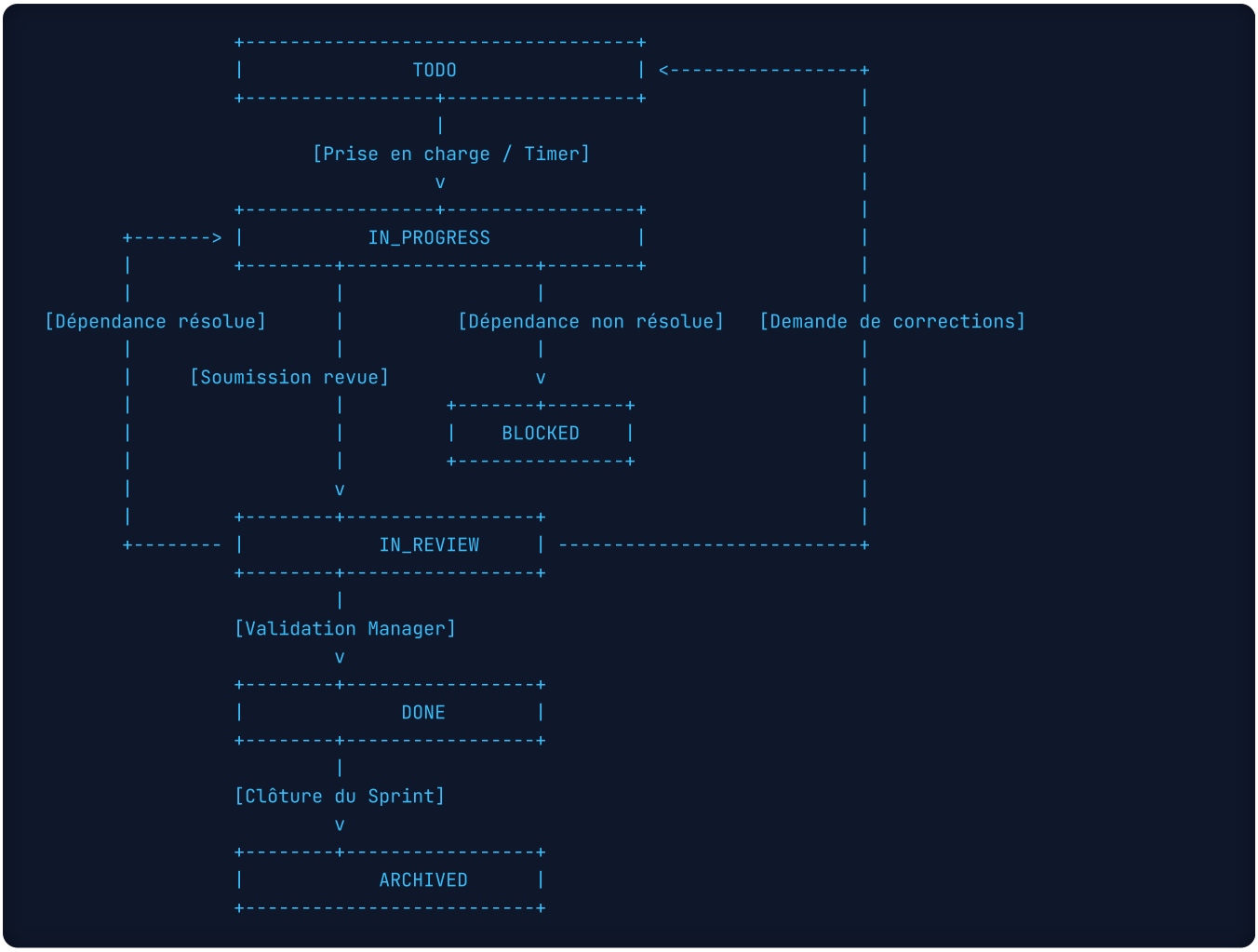
3. **Idempotence & Anti-Replay** : Dès qu'une invitation est consommée via l'endpoint `/projects/invitations/accept/:token`, son statut est basculé à `ACCEPTED` avec horodatage `acceptedAt = NOW()`. Toute tentative ultérieure de réutilisation est immédiatement rejetée avec une exception HTTP 400 (`Cette invitation a déjà été acceptée`).

---

5. Moteur Kanban & Algorithme Acyclique Gantt (DFS)

5.1 Machine à États Finis du Cycle de Vie d'une Tâche

Une tâche transite entre six états stricts :



5.2 Algorithme de Détection de Cycles dans le Graphe de Dépendances (DFS)

Dans un diagramme de Gantt, si la Tâche A dépend de la Tâche B, et la Tâche B dépend de la Tâche C, ajouter une dépendance de C vers A créerait une boucle infinie ($A \to B \to C \to A$) rendant impossible tout calcul de planification ou de dates critiques.

L'algorithme implémenté dans [dependencies.service.ts](file:///home/gaetan/Documents/GitHub/planner-pro/backend/src/projects/dependencies.service.ts#L13-L33) utilise une recherche récursive en profondeur (**Depth-First Search - DFS**) avec mémorisation des sommets visités :

```
/**
 * Recherche récursive DFS de chemin orienté entre deux sommets.
 * Complexité algorithmique : O(V + E) où V = sommets (tasks), E = arêtes (dependencies)
 */
private async hasPath(
  startTaskId: string,
  targetTaskId: string,
  visited: Set<string>,
): Promise<boolean> {
  // Condition d'arrêt 1 : Le chemin boucle sur le sommet cible (Cycle détecté)
  if (startTaskId === targetTaskId) return true;

  // Condition d'arrêt 2 : Branche déjà explorée sans succès (Évite de revisiter un sous-graphe)
  if (visited.has(startTaskId)) return false;
  visited.add(startTaskId);

  // Récupérer toutes les arêtes sortantes : tâches dont dépend startTaskId
  const dependencies = await this.prisma.taskDependency.findMany({
    where: { taskId: startTaskId },
    select: { dependsOnTaskId: true },
  });

  // Exploration en profondeur de chaque successeur
  for (const dep of dependencies) {
    if (await this.hasPath(dep.dependsOnTaskId, targetTaskId, visited)) {
      return true;
    }
  }

  return false;
}
```

- **Validation à l'insertion** : Avant d'insérer `(taskId, dependsOnTaskId)`, le service appelle `hasPath(dependsOnTaskId, taskId, new Set())`. Si un chemin existe déjà dans le sens inverse, l'insertion est immédiatement bloquée avec une erreur HTTP 400 : `"A cyclic dependency was detected. Task X cannot depend on Task Y"`.

---

6. Time-Tracking & Passerelle WebSockets Temps Réel

Le suivi du temps de travail s'appuie sur une architecture hybride combinant la précision d'une passerelle WebSocket bidirectionnelle et la robustesse de la persistance relationnelle.



6.1 Protocole & Gestion des Événements Temps Réel

- **Isolation des Salons (Rooms)** : À la connexion, le client rejoint automatiquement :

  1. Son salon personnel `user:${userId}` (pour synchroniser le chronomètre sur plusieurs onglets ou appareils de ce collaborateur).

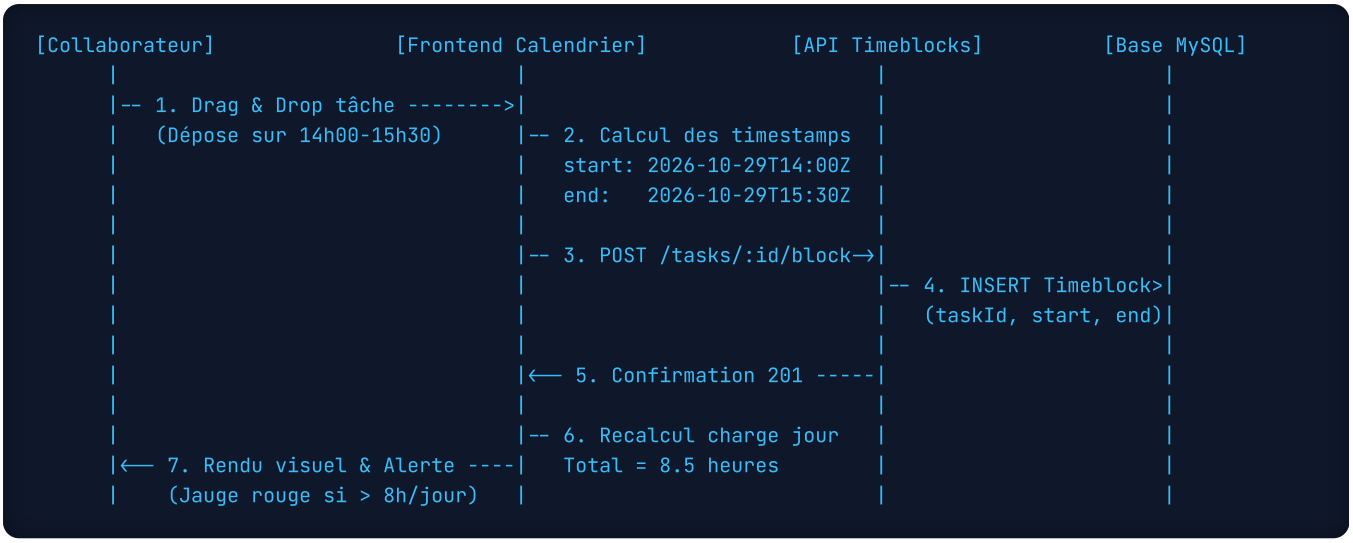
  2. Les salons de ses espaces `workspace:${workspaceId}` (pour notifier les collègues en temps réel de qui travaille sur quoi).

- **Adaptateur Distribué Redis** : Grâce à `@socket.io/redis-adapter`, si le backend est dimensionné sur 3 conteneurs distincts, un événement de démarrage de tracking émis sur le serveur A est instantanément relayé par Redis aux clients connectés sur les serveurs B et C.

---

7. Calendrier Interactif & Algorithme de Time-Blocking

Le module de calendrier concrétise la planification dans le temps réel en découpant la journée en créneaux bloqués (*Timeblocks*).



7.1 Algorithme de Détection de Surcharge Quotidienne

Lors de chaque modification de timeblock, le frontend et le backend exécutent le calcul de la somme des durées de tous les blocs d'un utilisateur sur la journée J :

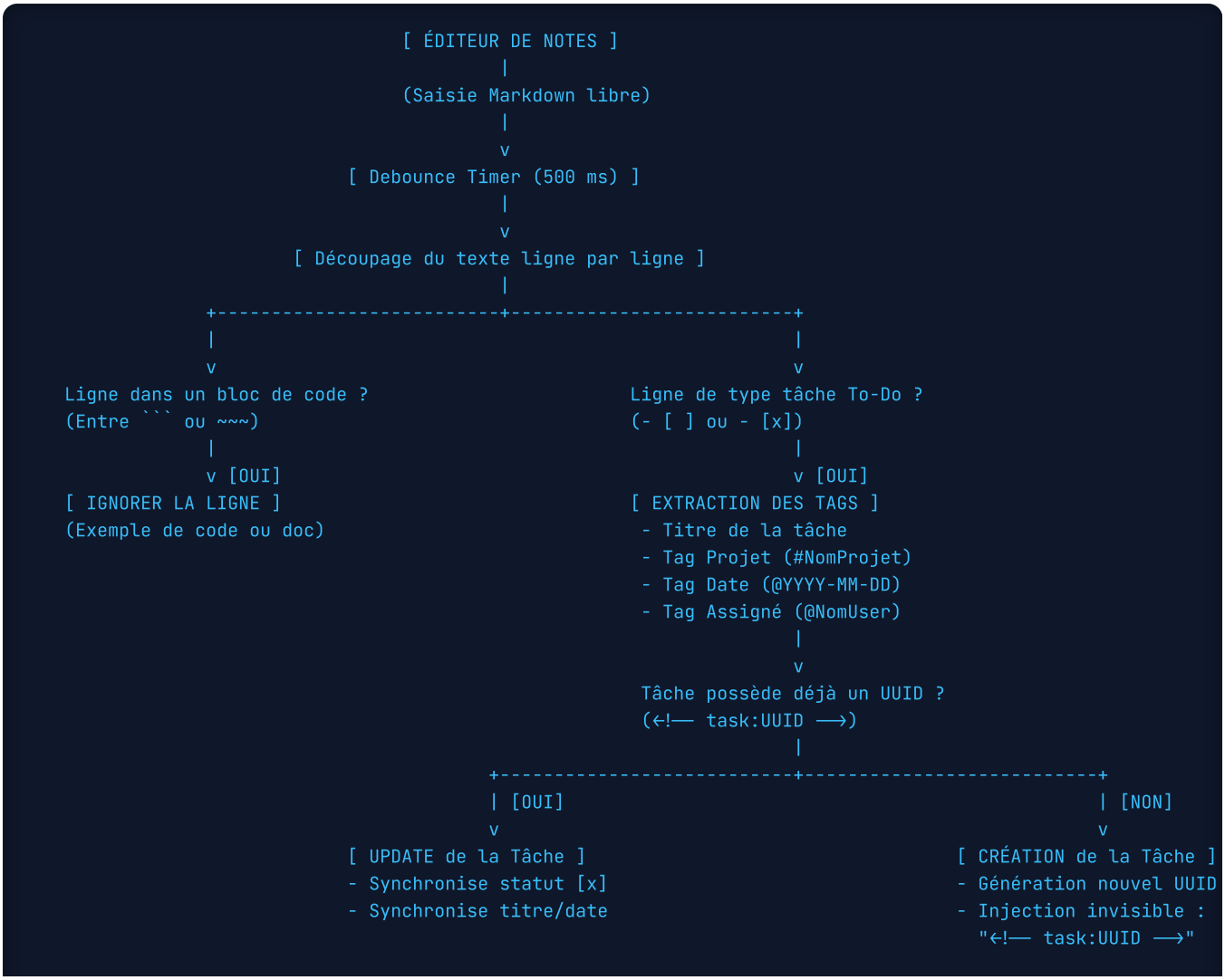
$$\text{Charge}(J) = \sum_{i \in \text{Timeblocks}(J)} (\text{endTime}_i - \text{startTime}_i)$$

- **Seuil Vert** : $\text{Charge}(J) \le 6\,\text{heures}$ (Capacité optimale de concentration).
- **Seuil Orange** : $6\,\text{heures} < \text{Charge}(J) \le 8\,\text{heures}$ (Journée pleine).
- **Seuil Rouge & Alerte Critique** : $\text{Charge}(J) > 8\,\text{heures}$ (Risque avéré de surmenage / surcharge cognitive).

---

8. Bloc-notes Intelligent & Moteur Lexical (Notes-to-Tasks)

L'éditeur de notes permet une rédaction continue en Markdown. Le moteur d'analyse lexicale en arrière-plan convertit les intentions écrites en tickets de projet réels sans interrompre la saisie.



8.1 Expressions Régulières & Préservation Lexicale

Le service [notes.service.ts](file:///home/gaetan/Documents/GitHub/planner-pro/backend/src/notes/notes.service.ts) utilise des patterns stricts :

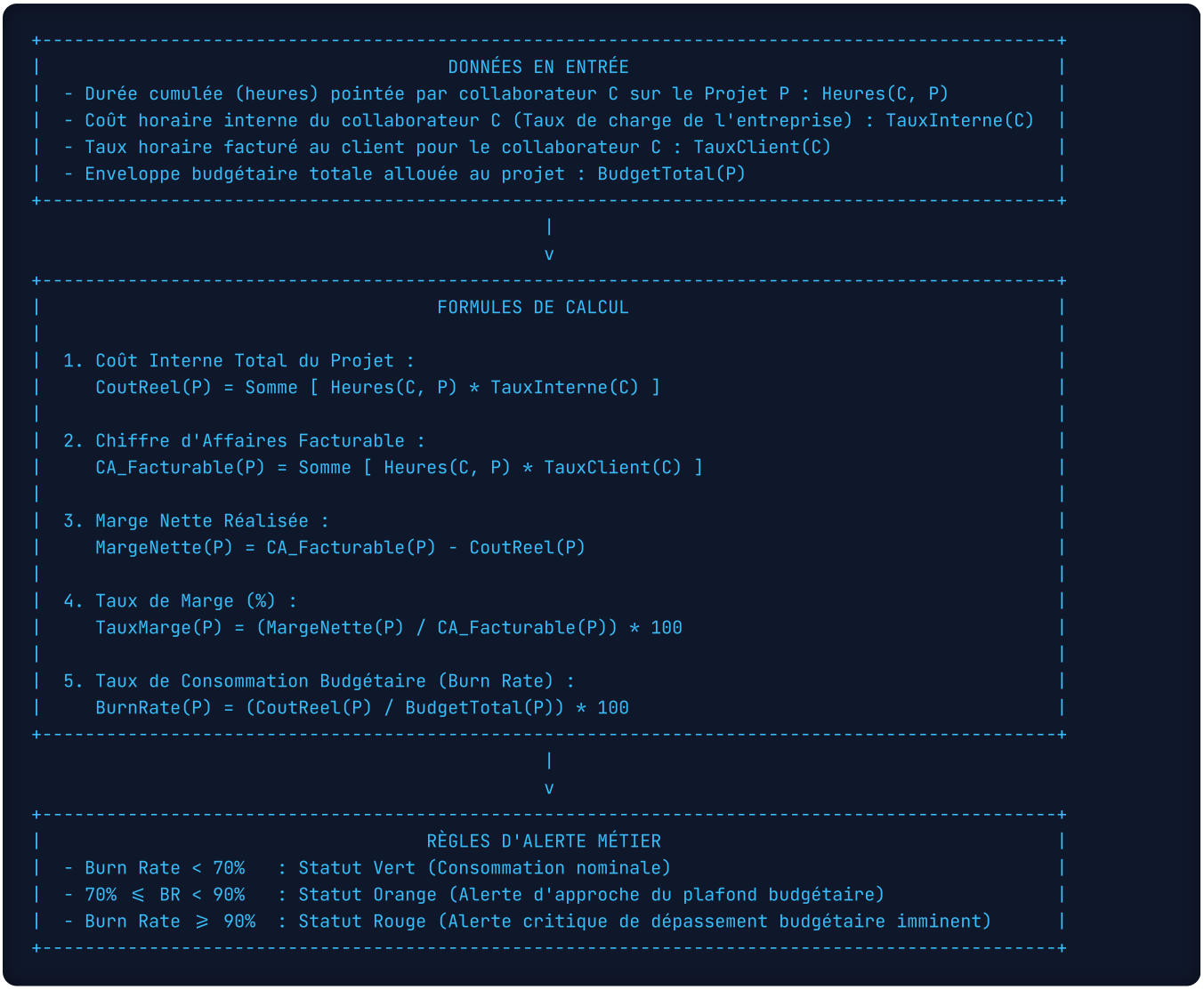
1. **Détection de case à cocher** : `/^(\s-\s\[([ xX])\])\s+(.*)$/`
2. **Extraction du tag de projet** : `/#([a-zA-Z0-9_\-]+)/`
3. **Extraction de la date d'échéance** : `/@(\d{4}-\d{2}-\d{2})/`
4. **Marqueur d'idempotence masqué** : `//`

- **Idempotence garantie** : En insérant le commentaire HTML masqué en fin de ligne Markdown, le texte reste parfaitement lisible pour l'utilisateur, tout en fournissant au moteur un identifiant déterministe pour synchroniser bidirectionnellement la case cochée et le statut Kanban de la tâche.

---

9. Moteur Financier, Budgets & Burn Rate Projet

Pour chaque projet, Planner-Pro croise les heures pointées issues du module Time-Tracking avec la grille tarifaire des collaborateurs pour dériver les métriques de rentabilité en temps réel.



---

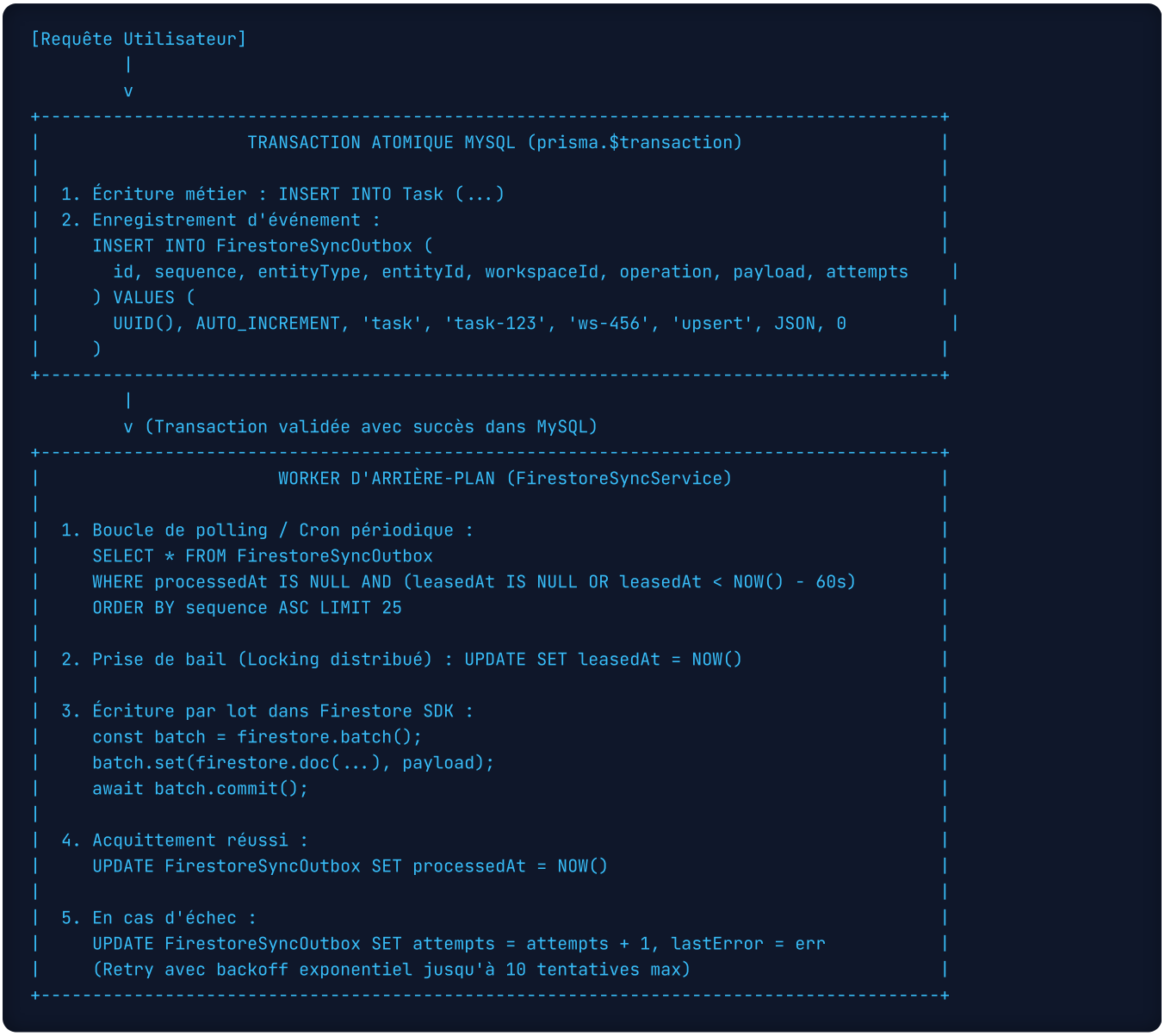
10. Synchronisation Hybride Asynchrone : Transactional Outbox vers Cloud Firestore

10.1 Problématique du Double-Write

Lorsqu'une entité (ex. une Tâche ou une Invitation) est créée, mettre à jour MySQL et appeler simultanément l'API distante Google Firestore expose l'architecture à des incohérences majeures si l'appel réseau vers Firestore échoue ou subit un timeout après que la transaction MySQL a été validée (commit).

10.2 Implémentation du Transactional Outbox Pattern

Pour éliminer ce problème, Planner-Pro applique le patron **Transactional Outbox** :



- **Purge automatique** : Un job planifié quotidien supprime automatiquement les événements de l'outbox dont le traitement (`processedAt`) remonte à plus de 7 jours, conservant une table légère et hautement performante.

---

11. Infrastructure de Production, Déploiement & Résilience

11.1 Topologie des Conteneurs Docker

La stack est déployée sur une instance Cloud (Oracle Linux ARM64 / Ubuntu 24.04 LTS) selon la configuration définie dans [docker-compose.prod.yml](file:///home/gaetan/Documents/GitHub/planner-pro/docker-compose.prod.yml) :

1. `planner-pro-production-backend` : Image Node.js 20 Alpine exécutant le binaire NestJS compilé.

2. `planner-pro-production-frontend` : Image Nginx Alpine servant l'application SPA React 18.

3. `planner-pro-production-db` : Serveur MySQL 8.0 avec volumes montés sur stockage persistant hôte (`/var/lib/mysql`).

4. `planner-pro-production-cache` : Serveur Redis 7.0 avec politique d'éviction mémoire `allkeys-lru`.

11.2 Stratégie de Sauvegarde & PRA (Plan de Reprise d'Activité)

- **Sauvegarde automatique** : Le script [scripts/backup-production.sh] (file:///home/gaetan/Documents/GitHub/planner-pro/scripts/backup-production.sh) effectue un dump cohérent de la base MySQL (`mysqldump --single-transaction --quick`), compresse l'archive avec `gzip` et chiffre le fichier résultant avec OpenSSL avant synchronisation distante.
- **Healthcheck & Auto-Healing** : Tous les conteneurs sont configurés avec `restart: unless-stopped` et des sondes de vitalité (`HEALTHCHECK`) testant les endpoints `/admin/health` toutes les 30 secondes.

Résumé de Conformité pour les Équipes d'Ingénierie

Ce document constitue la **spécification technique de référence (v2.0)** de Planner-Pro. Tout développement futur, ajout d'API ou refactorisation doit impérativement respecter les règles de non-régression, les patterns d'idempotence et les matrices d'habilitation détaillés dans ce dossier.